

Dark.Tyro — stripping PhotoGuard off the text-to-image editing pipeline

FIT5230 Malicious AI · Theme 2 (Text-to-Image) · Dark / attack side Echo Zhao · Nissa Cor lidea — Milestone 1, 28 August 2026

The IMPRESS baseline (NeurIPS 2023), repaired to run in 2026, end to end on one free GPU: `clean → PhotoGuard-protect → IMPRESS-purify → edit → measure`.

Run it on Kaggle, Accelerator = GPU T4 x2. Not the P100 — it is `sm_60`, and current PyTorch wheels ship no kernels for it (repair 11). Run top to bottom; each configuration zips itself the moment it finishes, so download the zip then.

Eleven repairs to the reference implementation

The published code does not run as-is. 1–6 are what it takes to get one successful run; 7–9 are bookkeeping faults that make *multi-configuration* results silently wrong; 10–11 are hardware limits. Each is marked `FIX n` in the code.

#	fault
1	<code>runwayml/stable-diffusion-inpainting</code> deleted from Hugging Face
2	mirror has no <code>revision="fp16"</code> branch
3	<code>sewar</code> imported by <code>pg_metric.py</code> , absent from <code>requirements.txt</code>
4	launcher shards across 4 GPUs
5	protect writes <code>adv_*</code> , purify reads <code>adapt_adv_*</code> — no bridge
6	<code>pg_generate.py</code> defaults <code>--diff_steps=50</code> while protect used <code>4</code> ; folder names are built from params, so
7	the purified folder name encodes no <code>pg_*</code> parameters — every configuration overwrites the last
8	bridge step used <code>glob(...)[0]</code> , an arbitrary pick once several runs exist
9	display cell hard-coded one configuration's parameters
10	<code>(200, 10)</code> exhausts 16 GB of GPU memory
11	Kaggle P100 is <code>sm_60</code> ; current torch wheels start at <code>sm_70</code>

Fixes 7 and 8 are the dangerous ones: they do not raise an error, they hand you the previous run's images under this run's label.

```
# 1 · Platform, device, paths, persistence. Everything below derives from
import os, sys, re, pathlib, subprocess, torch
```

```
BASE, PLATFORM = ((pathlib.Path('/kaggle/working'), 'kaggle') if os.path.isabs(
    else (pathlib.Path('/content'), 'colab') if os.path.isabs(
```

```

else (pathlib.Path.cwd(), 'local'))

# fp16 backward is unreliable off CUDA, so MPS/CPU run fp32 (see the pat
DEVICE, USE_FP32 = (('cuda:0', False) if torch.cuda.is_available()
                    else ('mps', True) if getattr(torch.backends, 'mps',
                    else ('cpu', True))

IMPRESS_DIR = BASE / 'Impress'
ROOT         = BASE / 'helen_face'      # must stay a SIBLING of Impress/:
ARCHIVE      = BASE / 'tyro_results'

SUBENV = dict(os.environ,
               PYTORCH_ENABLE_MPS_FALLBACK='1',
               PYTORCH_MPS_HIGH_WATERMARK_RATIO='0.0',
               # FIX 10: 2000 backwards through an unrolled UNet fragment
               # it fails with "tried to allocate 2 GiB" while memory is
               PYTORCH_CUDA_ALLOC_CONF='expandable_segments:True')

# FIX 11 · a torch wheel only contains kernels for the architectures it
# On an unsupported card everything loads, then the first real op dies t
# "no kernel image is available for execution on the device". Catch it h
GPU_OK = True
if DEVICE.startswith('cuda'):
    _tag = 'sm_%d%d' % torch.cuda.get_device_capability(0)
    GPU_OK = _tag in torch.cuda.get_arch_list()
    print(f'gpu      : {torch.cuda.get_device_name(0)} ({_tag}) · torch {t
    if not GPU_OK:
        print(f'!! this torch build has no kernels for {_tag}. Switch ac

# Persistence. Kaggle keeps /kaggle/working only if you Save Version or
# Colab wipes the VM on disconnect, so results go to Drive.
if PLATFORM == 'colab':
    try:
        from google.colab import drive; drive.mount('/content/drive')
        ARCHIVE = pathlib.Path('/content/drive/MyDrive/FIT5230_sweep')
    except Exception as e:
        print(f'!! Drive mount failed ({e}) – download tyro_results/ bef
ARCHIVE.mkdir(parents=True, exist_ok=True)
print(f'{PLATFORM} · {DEVICE} · {"fp32" if USE_FP32 else "fp16"} · resul

gpu      : Tesla T4 (sm_75) · torch 2.11.0+cu128
!! Drive mount failed (mount failed) – download tyro_results/ before disc
colab · cuda:0 · fp16 · results -> /content/tyro_results

```

```

# 2 · Clone IMPRESS, install only what is genuinely missing, patch the 2
#
# Do NOT `pip install -r Impress/requirements.txt` here. It lists torch,
# accelerate, datasets and tensorboard with no version pins; pip then fe
# plus the whole nvidia-* stack – several GB, ~20 min, and often a CUDA
# Kaggle and Colab already ship those. Install the small pure-Python gap

```

```

import importlib.util

# sewar: FIX 3, imported by pg_metric.py but missing from requirements.txt
# gdown: ours – the dataset is a Google Drive link in the README, not a
# ftfy: in requirements.txt; a transitive need of the CLIP tokenizer, i
NEEDED = {'sewar':'sewar', 'lpips':'lpips', 'gdown':'gdown', 'ftfy':'ftf
missing = [pkg for mod, pkg in NEEDED.items() if importlib.util.find_spe
if missing:
    subprocess.run(f'pip install -q --no-deps {" ".join(missing)}', shel
print('installed:', missing or 'nothing needed')

if not IMPRESS_DIR.exists():
    subprocess.run(f'git clone -q --depth 1 https://github.com/AAAAAAsuk
                    shell=True, check=True)

# Patched in Python, not sed: BSD sed on macOS rejects `sed -i 's/.../.../
n = 0
for p in sorted(IMPRESS_DIR.glob('*.py')):
    s = orig = p.read_text()
    s = s.replace('runwayml/stable-diffusion-inpainting',
                  'stable-diffusion-v1-5/stable-diffusion-inpainting')
    s = re.sub(r'^[ \t]*revision="fp16",[ \t]*\r?\n', '', s, flags=re.M)
    if USE_FP32:
        s = s.replace('torch.float16', 'torch.float32').replace('.half()
    if s != orig:
        p.write_text(s); n += 1
print(f'cloned + patched {n} files')

installed: ['sewar', 'lpips', 'ftfy']
cloned + patched 6 files

```

```

# 3 · Hugging Face (non-blocking) and the Helen face data.
# SD 1.5 is public, so anonymous download normally works – this only pic
# is already there. `notebook_login()` is deliberately not called: it bl
# for a paste, which looks exactly like a hung cell.
try:
    from huggingface_hub import login, whoami
    tok = os.environ.get('HF_TOKEN') or os.environ.get('HUGGINGFACE_HUB_
    if not tok and PLATFORM == 'kaggle':
        from kaggle_secrets import UserSecretsClient
        tok = UserSecretsClient().get_secret('HF_TOKEN') # Add-ons ->
    if tok: login(token=tok, add_to_git_credential=False)
    print('hf user:', whoami().get('name'))
except Exception:
    print('hf: anonymous (fine – SD 1.5 is public; only revisit this on

# Dataset: the Google Drive link from the IMPRESS README, unzipped as a
import zipfile, gdown
N_IMAGES = 2 # >= 2, or pg_metric's torch.std() returns nan and the
zip_path = BASE / 'helen_face_dataset.zip'
if not zip_path.exists():

```

```

gdown.download(id='16xISe7M_DlSqM2Zf2lWI4JJXcdsDEPsl', output=str(z
if not (R00T / 'clean').exists():
    R00T.mkdir(exist_ok=True)
    with zipfile.ZipFile(zip_path) as z: z.extractall(R00T)

keep = set(sorted(os.listdir(R00T / 'clean'))[:N_IMAGES]) # clean/ and
for sub in ('clean', 'mask'):
    for f in os.listdir(R00T / sub):
        if f not in keep: os.remove(R00T / sub / f)
print(f'{N_IMAGES} face(s):', sorted(keep))

hf: anonymous (fine – SD 1.5 is public; only revisit this on a 401)
2 face(s): ['1233476865_1.png', '1525918600_1.png']

```

✓ Why the grid is `pg_iters` × `pg_grad_reps`

From `pg_mask_diff_helen.py`:

```

grad_normalized = grad.detach() / (grad_norm + 1e-10) # unit-length direc
X_adv = X_adv - grad_normalized * actual_step_size # step_size = 1 ->
d_x_norm = torch.renorm(d_x, p=2, dim=0, maxnorm=eps) # projected back in

```

Every PGD step moves the image exactly `pg_step_size` = 1.0 in L2 and is then projected back inside a ball of radius `pg_eps` = 16. **From the centre you hit the wall after ~16 steps.** Everything after that walks *around* the surface, not further out.

The leash. `pg_eps` is the length of the dog's leash; `pg_iters` is how long the dog runs. Once the leash is taut, running longer does not get the dog one inch further from the post — it only lets it find a better spot along the arc.

So the grid is not "more is better". `(40, 10)` and `(200, 2)` cost the **same compute**, and comparing them asks something the paper does not: *at a fixed budget, is it better to take more steps or to estimate each step better?*

lever	what it changes		cost
<code>pg_eps</code> 16 → 32	the size of the ball — the real ceiling on shield strength		free (type=int)
<code>pg_grad_reps</code> 2 → 10	averages 10 stochastic gradients instead of 2 → a truer direction (EOT)		linear, ×5
<code>diff_steps</code> 4 → 8	how much of the diffusion process the attack unrolls		linear, and the me
<code>pg_iters</code> 40 → 200	steps taken <i>inside</i> the ball		linear, and satura

`(200, 10)` exhausts 16 GB and is omitted by design: it spends 5× the compute of `(40, 10)` on the lever that had already run out. Dropping `diff_steps` to rescue

that one point would mean a grid where one cell used a different attack, which is not a grid.

Trap: `--attack_type=linf` is a no-op here — `super_linf` clamps to $X \pm \text{eps}$ on images in `[-1, 1]`, so at `pg_eps=16` the constraint never binds. Stay on `l2`.

```
# 4 · FIX 9 – the one place parameters live. Every path, command and lab
#   f-string off this dict, so no two cells can drift apart.
#   Values follow the reference run in Impress/scripts/new/pg_mask_dif
#   pur_iters (see the note) – argparse defaults differ and are NOT wh
PARAMS = dict(
    # --- PhotoGuard: the DEFENCE we attack. Fixed across the sweep. ---
    attack_type = 'l2',      # linf never binds at eps=16 on [-1,1] ima
    pg_eps      = 16,        # L2 budget = the ceiling on shield streng
    pg_step_size = 1,        # each PGD step moves L2 = exactly 1.0
    pg_eta      = 1,        # DDIM eta inside the attack; 1 = stochast
    diff_steps  = 4,        # steps the attack unrolls – the memory dr
    guidance    = 7.5,
    seed        = 0,        # fixed -> the 3-panel comparison is contr

    # --- IMPRESS: OUR attack. Fixed, so pg_* is the only variable. ---
    pur_eps     = 0.1,      # free LPIPS allowance before the stay-clo
    pur_iters   = 100,      # NOTE: the reference script uses 1000. 10
                                # and is what M1 ran; raising it is the fi

    pur_lr      = 0.005,
    pur_alpha   = 0.01,
    pur_noise   = 0.05,

    # --- The evaluation edit. Fixed, or the comparison means nothing. -
    prompt      = 'a person in an airplane',
    test_guidance = 7.5,
    test_diff_steps = 50,
)

CONFIGS = [(40, 2), (200, 2)]    # cheapest end-to-end check + stronges
# CONFIGS = [(40, 2)]           # plumbing check only, ~5 min
# CONFIGS = [(40, 2), (40, 10), (200, 2)] # the wider grid; (200,10) o

# Rough ETA, calibrated on our own measured (40,2) = 4.9 min at N_IMAGES
# attack is backward-heavy through an unrolled UNet, so it tracks memory
# – a T4 (320 GB/s) is ~2.2x slower than a P100 (732 GB/s). Key the scal
_SCALE = {'P100':1.0, 'T4':2.2, 'V100':0.6, 'A100':0.35, 'L4':1.5, 'P4':
_card = torch.cuda.get_device_name(0) if DEVICE.startswith('cuda') else
_s     = next((v for k, v in _SCALE.items() if k in _card), 39.0 if DEVI
eta     = lambda i, g: (i * g * N_IMAGES * 0.71/60) * _s + 3.0

print(f'{len(CONFIGS)} config(s) x {N_IMAGES} images on {_card} (x{_s} v
for i, g in CONFIGS: print(f'  ({i:>3}, {g:>2}) ~{eta(i, g):5.1f} min')
print(f'  {"total":>9} ~{sum(eta(i, g) for i, g in CONFIGS):5.1f} min')
```

```
2 config(s) x 2 images on Tesla T4 (x2.2 vs our P100 baseline)
( 40,  2) ~ 7.2 min
(200,  2) ~ 23.8 min
      total ~ 31.0 min
```

✓ The harness

Paths are rebuilt exactly as the source builds them — including its quirk of using `re.sub("pur", "pur_diff", ...)`, which rewrites **every** occurrence of `pur` in the string, not just the first. Guessing that folder name by hand is how you read an empty directory and conclude the attack failed.

Three properties the reference scripts do not have:

- **Images are archived before metrics are parsed.** The images are the deliverable; a metric-parsing failure must never discard a finished 15-minute run.
- **A heartbeat.** `subprocess.run(..., stdout=PIPE)` buffers until exit, so a working stage and a hung one look identical. A background thread prints elapsed time every 60 s.
- **Stale purified folders are wiped** before each configuration (FIX 7).

```
# 5 · Path helpers, stage runner, metric parser, and the per-configuration
import shutil, json, time, threading
P = PARAMS

# FIX 8: rebuild these from parameters – never glob for them.
def adv_dir(i, g):
    return (f"{ROOT}/adv_{P['attack_type']}_eps_{P['pg_eps']}_step_{P['pg_
        f'_iter{i}grad_reps{g}_eta_{P['pg_eta']}_diff_steps_{P['diff_s
        f'_guidance_{P['guidance']}_seed_{P['seed']}]}"
adapt_dir    = lambda i, g: adv_dir(i, g).replace('/adv_', '/adapt_adv_')
adv_diff_dir = lambda i, g: re.sub('adv', 'adv_diff', adv_dir(i, g))
pur_dir      = lambda: (f"{ROOT}/pur_eps_{P['pur_eps']}_pur_iters_{P['pur_
        f'_pur_alpha_{P['pur_alpha']}_pur_noise_{P['pur_no
pur_diff_dir = lambda: re.sub('pur', 'pur_diff', pur_dir()) # mirrors
tag_of       = lambda i, g: f'iter{i}_grad{g}_eps_{P["pg_eps"]}'

# FIX 4: --parallel_index=-1 disables the 4-GPU sharding the launcher sc
PG  = (f"--attack_type={P['attack_type']} --pg_eps={P['pg_eps']} --pg_st
        f"--pg_eta={P['pg_eta']} --parallel_index=-1 --device={DEVICE}")
PUR = (f"--pur_eps={P['pur_eps']} --pur_iters={P['pur_iters']} --pur_lr=
        f"--pur_alpha={P['pur_alpha']} --pur_noise={P['pur_noise']}")

def sh(cmd, heartbeat=60):
    print(f'$ {cmd}', flush=True)
    t0, stop = time.time(), threading.Event()
```

```

def _beat():
    while not stop.wait(heartbeat):
        print(f'    ... {(time.time()-t0)/60:.1f} min elapsed', flush=True)
    threading.Thread(target=_beat, daemon=True).start()
try:
    r = subprocess.run(f'{sys.executable} -u {cmd}', shell=True, cwd=
                        env=SUBENV, stdout=subprocess.PIPE, stderr=
finally:
    stop.set()
print(r.stdout[-3000:], f'\n    [{(time.time()-t0)/60:.1f} min]', flu
return r.stdout

# NumPy 2 prints {'ssim': [np.float64(0.53), ...]}; ast.literal_eval rej
# np.float64(...) parses as a CALL, not a literal. Kaggle ships NumPy 2,
# which is why this worked in one place and not the other. Pull the numb
_NUM = r'(?:(?:np\.|w+)(\s*)?(-?(?:\d+\.?\d*(?:[eE][+-]?\d+)?|nan|inf))\s*
_PAIR = re.compile(r''(\w+)\s*:\s*\s*(\s*' + _NUM + r'\s*,\s*' + _NUM + r

def parse_metric(out):
    return {m.group(1): {k: [float(a), float(b)] for k, a, b in _PAIR.fi
            for m in (re.match(r'^(\w+)\s*:\s*(\s*\s*)\s*$', l.strip()))

def run_config(iters, reps):
    tag = tag_of(iters, reps)
    print('=' * 70, f'\n    CONFIG {tag}    (expect ~{eta(iters, reps):.0f}
    t0 = time.time()

    # FIX 7: the purified folder name carries no pg_* parameters, so eve
    # writes to the SAME directory, with no skip-if-exists guard and no
    # folder means this run silently reports the PREVIOUS configuration'
    for stale in (pur_dir(), pur_diff_dir()):
        if os.path.exists(stale): shutil.rmtree(stale)

    # (a) protect – PhotoGuard. Skips images already done, so a re-run r
    sh(f'pg_mask_diff_helen.py {PG} --pg_iters={iters} --pg_grad_reps={r
        f'--diff_steps={P["diff_steps"]} --guidance={P["guidance"]}')

    # (b) FIX 5: bridge the broken folder contract between protect and p
    src, dst = adv_dir(iters, reps), adapt_dir(iters, reps)
    assert os.path.isdir(src), f'protect stage produced nothing at {src}
    if os.path.exists(dst): shutil.rmtree(dst)
    shutil.copytree(src, dst)

    # (c) purify – IMPRESS, our baseline attack.
    sh(f'pg_mask_pur_helen.py {PG} --pg_iters={iters} --pg_grad_reps={re
        f'--diff_steps={P["diff_steps"]} {PUR}')

    # (d) edit all three variants. FIX 6: diff_steps forced through, or
    common = (f'{PG} --pg_iters={iters} --pg_grad_reps={reps} --diff_ste
        f'--guidance={P["guidance"]} {PUR} --test_guidance={P["tes
        f'--test_diff_steps={P["test_diff_steps"]} --prompt="{P["p

```

```

sh(f'pg_generate.py {common}')

# Archive FIRST, under a stamped tag – the other half of FIX 7: the
# artefact that records which purification belongs to which protecti
# finished configuration is downloadable immediately instead of afte
dest = ARCHIVE / tag; dest.mkdir(parents=True, exist_ok=True)
for name, path in [('clean', f'{ROOT}/clean'), ('protected', adv_dir
    ('purified', pur_dir()), ('edit_clean', f'{ROOT}/
    ('edit_protected', adv_diff_dir(iters, reps)), ('
    if os.path.isdir(path): shutil.copytree(path, dest / name, dirs_
(dest / 'params.json').write_text(json.dumps(**P, 'pg_iters': iters
zp = shutil.make_archive(str(ARCHIVE / tag), 'zip', root_dir=str(des
print(f'[saved] {zp} ({os.path.getsize(zp)/1e6:.1f} MB) <- DOWNLOADA

# Metrics last, and never fatal: the images are the deliverable, met
raw = sh(f'pg_metric.py {common}')
(dest / 'metric_stdout.txt').write_text(raw)
print(f'[done] {tag} ({(time.time()-t0)/60:.1f} min)', flush=True)
return parse_metric(raw)

print('harness ready ·', pur_dir())

harness ready · /content/helen_face/pur_eps0.1_pur_iters100_pur_lr0.005_p

```

```

# 6 · RUN. Archived after each configuration, so a disconnect costs only
assert GPU_OK, 'this torch build has no kernels for this GPU – switch to

RESULTS = {}
for iters, reps in CONFIGS:
    try:
        RESULTS[(iters, reps)] = run_config(iters, reps)
    except Exception as e:
        # one dead configuration
        print(f'!! ({iters}, {reps}) failed: {type(e).__name__}: {e}')
        RESULTS[(iters, reps)] = None
        if DEVICE.startswith('cuda'): torch.cuda.empty_cache()

(ARCHIVE / 'scores.json').write_text(
    json.dumps({f'{i}_{g}': v for (i, g), v in RESULTS.items()}), indent=
print('\ndone ->', ARCHIVE / 'scores.json')

```

```

=====
CONFIG iter40_grad2_eps16 (expect ~7 min)
=====
$ pg_mask_diff_helen.py --attack_type=l2 --pg_eps=16 --pg_step_size=1 --
... 1.0 min elapsed
... 2.0 min elapsed
... 3.0 min elapsed
... 4.0 min elapsed
... 5.0 min elapsed
... 6.0 min elapsed
... 7.0 min elapsed

```



```

... 8.0 min elapsed
, 5.28s/it]
AVG Loss: 1275.200: 48%|██████████| 19/40 [01:39<01:50, 5.28s/it]
AVG Loss: 1427.078: 48%|██████████| 19/40 [01:45<01:50, 5.28s/it]
AVG Loss: 1427.078: 50%|██████████| 20/40 [01:45<01:45, 5.28s/it]
AVG Loss: 1490.676: 50%|██████████| 20/40 [01:50<01:45, 5.28s/it]
AVG Loss: 1490.676: 52%|██████████| 21/40 [01:50<01:40, 5.28s/it]
AVG Loss: 1461.312: 52%|██████████| 21/40 [01:55<01:40, 5.28s/it]
AVG Loss: 1461.312: 55%|██████████| 22/40 [01:55<01:35, 5.29s/it]
AVG Loss: 1329.510: 55%|██████████| 22/40 [02:01<01:35, 5.29s/it]
AVG Loss: 1329.510: 57%|██████████| 23/40 [02:01<01:29, 5.29s/it]
AVG Loss: 1293.304: 57%|██████████| 23/40 [02:06<01:29, 5.29s/it]
AVG Loss: 1293.304: 60%|██████████| 24/40 [02:06<01:24, 5.28s/it]
AVG Loss: 1310.771: 60%|██████████| 24/40 [02:11<01:24, 5.28s/it]
AVG Loss: 1310.771: 62%|██████████| 25/40 [02:11<01:19, 5.28s/it]
AVG Loss: 1313.551: 62%|██████████| 25/40 [02:16<01:19, 5.28s/it]
AVG Loss: 1313.551: 65%|██████████| 26/40 [02:16<01:13, 5.28s/it]
AVG Loss: 1470.177: 65%|██████████| 26/40 [02:22<01:13, 5.28s/it]
AVG Loss: 1470.177: 68%|██████████| 27/40 [02:22<01:08, 5.28s/it]
AVG Loss: 1337.351: 68%|██████████| 27/40 [02:27<01:08, 5.28s/it]
AVG Loss: 1337.351: 70%|██████████| 28/40 [02:27<01:03, 5.28s/it]
AVG Loss: 1131.447: 70%|██████████| 28/40 [02:32<01:03, 5.28s/it]
AVG Loss: 1131.447: 72%|██████████| 29/40 [02:32<00:58, 5.28s/it]
AVG Loss: 1288.543: 72%|██████████| 29/40 [02:38<00:58, 5.28s/it]
AVG Loss: 1288.543: 75%|██████████| 30/40 [02:38<00:52, 5.29s/it]
AVG Loss: 1012.252: 75%|██████████| 30/40 [02:43<00:52, 5.29s/it]
AVG Loss: 1012.252: 78%|██████████| 31/40 [02:43<00:47, 5.28s/it]
AVG Loss: 1212.767: 78%|██████████| 31/40 [02:48<00:47, 5.28s/it]
AVG Loss: 1212.767: 80%|██████████| 32/40 [02:48<00:42, 5.28s/it]
AVG Loss: 1241.605: 80%|██████████| 32/40 [02:53<00:42, 5.28s/it]
AVG Loss: 1241.605: 82%|██████████| 33/40 [02:53<00:37, 5.29s/it]
AVG Loss: 1631.813: 82%|██████████| 33/40 [02:59<00:37, 5.29s/it]
AVG Loss: 1631.813: 85%|██████████| 34/40 [02:59<00:31, 5.29s/it]
AVG Loss: 1097.730: 85%|██████████| 34/40 [03:04<00:31, 5.29s/it]
AVG Loss: 1097.730: 88%|██████████| 35/40 [03:04<00:26, 5.28s/it]
AVG Loss: 1426.825: 88%|██████████| 35/40 [03:09<00:26, 5.28s/it]
AVG Loss: 1426.825: 90%|██████████| 36/40 [03:09<00:21, 5.28s/it]
AVG Loss: 987.860: 90%|██████████| 36/40 [03:14<00:21, 5.28s/it]
AVG Loss: 987.860: 92%|██████████| 37/40 [03:14<00:15, 5.28s/it]
AVG Loss: 1165.381: 92%|██████████| 37/40 [03:20<00:15, 5.28s/it]
AVG Loss: 1165.381: 95%|██████████| 38/40 [03:20<00:10, 5.28s/it]
AVG Loss: 982.235: 95%|██████████| 38/40 [03:25<00:10, 5.28s/it]
AVG Loss: 982.235: 98%|██████████| 39/40 [03:25<00:05, 5.29s/it]
AVG Loss: 982.809: 98%|██████████| 39/40 [03:30<00:05, 5.29s/it]
AVG Loss: 982.809: 100%|██████████| 40/40 [03:30<00:00, 5.28s/it]
AVG Loss: 982.809: 100%|██████████| 40/40 [03:30<00:00, 5.27s/it]

```

✓ Reading the numbers

⚠ `pg_metric.py` compares the *edited* images against the *edited clean* image — not against the original photograph. From the source:

```

clean_image = load_image(diff_dir_clean, image_name) # the EDITED clean image
image_quality_metrics(clean_image, adv_image, adv_score_dict)
image_quality_metrics(clean_image, pur_image, pur_score_dict)

```

So these numbers answer "*did the editing pipeline behave normally?*" — **not** "*is the photo intact?*" Two different things are called SSIM in this project and conflating them is the easiest mistake available here.

- **adv** — edited-protected vs edited-clean. **Lower = the shield worked better.**
- **pur** — edited-purified vs edited-clean. **Higher = our purification worked better.**
- **$R_{\text{pipe}} = (\text{pur} - \text{adv}) / (1 - \text{adv})$** — the share of the shield's damage that purification undid. SSIM = 1.0 is by construction "the pipeline behaved as if the photo had never been protected", which gives a principled denominator with no third measurement. It is a *pipeline* rate, **not** an edit-success rate: it says nothing about whether the output matches the prompt. That second measure is M2 work.

```

# 7 · Results, R_pipe, and the sign test.
import pandas as pd

rows = [dict(config=f'{i}:{g}', units=i*g, metric=m,
            adv=round(sc['adv'][m][0], 4), pur=round(sc['pur'][m][0], 4),
            recovery=round(sc['pur'][m][0] - sc['adv'][m][0], 4))
        for (i, g), sc in RESULTS.items() if sc
        for m in ('ssim', 'psnr', 'vifp')
        if m in sc.get('adv', {}) and m in sc.get('pur', {})]
df = pd.DataFrame(rows).sort_values(['metric', 'units']) if rows else pd
display(df)

if len(df):
    ss = df[df.metric == 'ssim'].copy()
    ss['R_pipe'] = ((ss.pur - ss.adv) / (1 - ss.adv) * 100).round(2)
    print('\nPIPELINE RESTORATION RATE R_pipe = (pur - adv) / (1 - adv)')
    display(ss[['config', 'units', 'adv', 'pur', 'recovery', 'R_pipe']])

    # With two images per configuration any single metric can wander. But
    # are three different instruments on the same pair of pictures: if a
    # SIGN, something real moved. If they disagree, the recovery is indi
    # zero — and saying so is a result, not a failure to get one.
    print('\nDirection agreement across the three metrics:')
    for cfg in df.config.unique():
        sub = df[df.config == cfg]
        one = len({r.recovery > 0 for _, r in sub.iterrows()}) == 1
        print(f'    {cfg:>8} ' + ' '.join(f'{r.metric} {"+" if r.recovery > 0 else "-"} '
            + (' -> all three agree: real movement' if one else 'disagree')))

```

	config	units	metric	adv	pur	recovery
1	40:2	80	psnr	13.7849	16.7384	2.9535
4	200:2	400	psnr	14.3218	15.6560	1.3342
0	40:2	80	ssim	0.5506	0.6237	0.0731
3	200:2	400	ssim	0.5454	0.5455	0.0002
2	40:2	80	vifp	0.2004	0.2247	0.0244
5	200:2	400	vifp	0.1753	0.1813	0.0061

PIPELINE RESTORATION RATE $R_{\text{pipe}} = (\text{pur} - \text{adv}) / (1 - \text{adv}), \%$

	config	units	adv	pur	recovery	R_{pipe}
0	40:2	80	0.5506	0.6237	0.0731	16.27
3	200:2	400	0.5454	0.5455	0.0002	0.02

Direction agreement across the three metrics:

40:2 psnr + ssim + vifp + → all three agree: real movement
 200:2 psnr + ssim + vifp + → all three agree: real movement

```
# 8 · The headline figure, then a check that the results are somewhere d
# Same seed, mask and prompt in every panel – pg_generate.py re-seed
# immediately before each generation – so every visible difference c
# IMAGE ALONE. That is what makes this a controlled comparison.
import matplotlib.pyplot as plt, glob
from PIL import Image

done = [c for c in CONFIGS if (ARCHIVE / tag_of(*c) / 'edit_clean').is_d
assert done, 'nothing archived yet – run cell 6 first'
name = sorted(os.listdir(ARCHIVE / tag_of(*done[0]) / 'edit_clean'))[0]

fig, axes = plt.subplots(len(done), 3, figsize=(12, 4*len(done)), squeeze
for r, (i, g) in enumerate(done):
    for c, (title, sub) in enumerate([('edited CLEAN', 'edit_clean'),
                                      ('edited PROTECTED', 'edit_protect
                                      ('edited PURIFIED', 'edit_purified

        f = ARCHIVE / tag_of(i, g) / sub / name
        if f.exists(): axes[r][c].imshow(Image.open(f))
        axes[r][c].set_xticks([]); axes[r][c].set_yticks([])
        if r == 0: axes[r][c].set_title(title, fontsize=13)
    axes[r][0].set_ylabel(f'iters={i}\ngrad_reps={g}', fontsize=11)
fig.suptitle(f'Shield-strength sweep · "{P["prompt"]}" · seed {P["seed"]}')
fig.tight_layout(); fig.savefig(ARCHIVE / 'sweep_grid.png', dpi=110, bbo

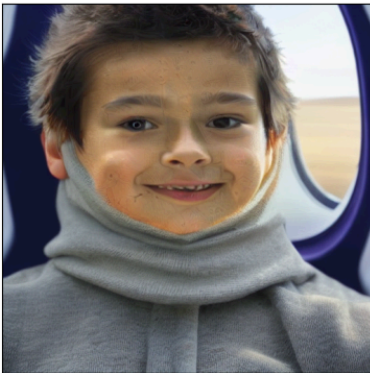
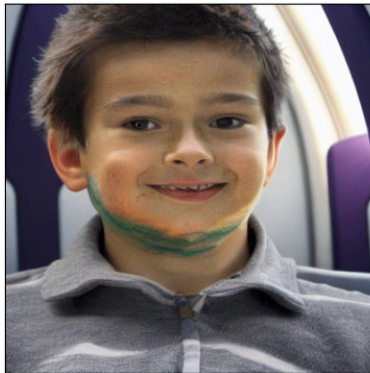
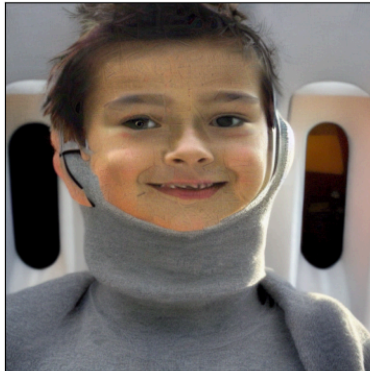
zips = sorted(glob.glob(str(ARCHIVE / '*.zip')))
print(f'\n{len(zips)} zip(s) in {ARCHIVE}:', [f'{os.path.basename(z)} ({
```

```

if PLATFORM == 'kaggle':
    print('NOT YET SAFE – download them from the Output panel, or Save V

```

Shield-strength sweep · "a person in an airplane" · seed 0 (identical in every panel)

	edited CLEAN	edited PROTECTED	edited PURIFIED
iters=40 grad_reps=2			
iters=200 grad_reps=2			

2 zip(s) in /content/tyro_results: ['iter200_grad2_eps16.zip (23 MB)', 'i

Limitations

Stated plainly, because a claim we cannot evidence costs more than a modest result.

1. **Scale.** Two faces per configuration — enough for `pg_metric` to report a real standard deviation instead of `nan`, and enough for a controlled visual comparison, but not enough to rank two configurations against each other. We do not attempt that ranking.
2. **How to read it at n = 2.** Never trust one metric on two images; trust whether PSNR, SSIM and VIF agree on the **sign**. Agreement means something moved,

disagreement means it is inside the noise — and saying so is a result.

3. **Run-to-run spread.** The RNG is seeded, but fp16 kernel selection and accumulation order are not fixed across GPU model and torch build. Repeat runs of identical parameters differ by about ± 5 percentage points of `R_pipe`. At $n = 2$ the noise is the size of the effect.
4. **`pg_eps` is untested.** We vary `pg_iters` and `pg_grad_reps` only. Both 400-unit points landing near zero is *consistent* with `pg_iters` saturating, but equally consistent with both shields simply winning. This experiment cannot separate those, and we do not claim it.
5. **`pur_iters = 100`, where the reference script uses 1000.** Our own attack ran at a tenth of the paper's optimisation budget. That is a candidate explanation for the small recovery and the first thing M2 tests.
6. **`pg_metric`'s SSIM compares edited images to the edited clean image**, not to the original photograph. `R_pipe` is built on that question only.
7. **Why the recovery is small is an open question**, and we treat it as one. The diagnostics are deliberately outside this notebook; they are M2 work and we quote no number from them.

What M2 adds

- ≥ 5 faces (the cheapest way to shrink the error bar), and a `pg_eps` sweep `16 → 24 → 32` — the lever that raises the shield's ceiling, which `pg_iters` does not.
- `pur_iters` `100 → 1000` and `pur_eps` `0.1 → 0.3`: does our purifier move further than the VAE round-trip floor once it is given the paper's budget?
- A quantitative **edit-success** rate (CLIPScore against the prompt) beside `R_pipe`, and **LPIPS** beside SSIM.
- The cheap-purifier ablation and the frequency-targeted filter, on one fidelity-vs-success chart.